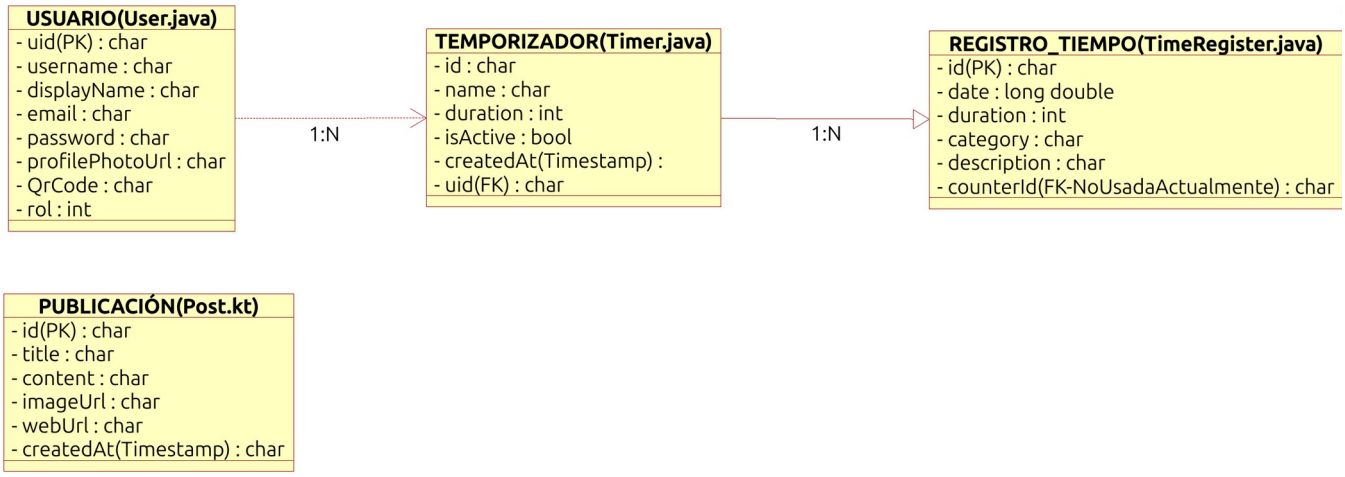
# Diagrama Entidad - Relación

# Temporis:

## Una herramienta accesible para el control y la optimización de nuestro tiempo

Kevin Zamora Amela

# 1. Diagrama Entidad – Relación




## 1.1 Introducción al Modelo de Datos

El presente documento detalla el **Diagrama Entidad-Relación (E-R)** del proyecto **Temporis**. El diseño de este modelo se ha realizado de forma coherente con la arquitectura de clases implementada en el repositorio del proyecto, asegurando que cada entidad responda a una funcionalidad real de la aplicación.

# 2. Justificación Tecnológica: NoSQL (Firebase Firestore) vs. Relacional (SQL)

Para el desarrollo de Temporis, se ha optado por implementar una base de datos **NoSQL orientada a documentos** mediante **Firebase Cloud Firestore**, descartando opciones relacionales tradicionales, como MySQL con PhpMyAdmin. Esta decisión se fundamenta en los siguientes pilares estratégicos, enfocados hacia el crecimiento de la aplicación:

- **Escalabilidad Dinámica:** A diferencia de MySQL, donde los esquemas son rígidos, Firestore permite una estructura flexible. Esto facilita la **expansión futura** de la app (por ejemplo, añadir nuevos atributos a los temporizadores) sin necesidad de realizar migraciones de esquemas complejas que podrían interrumpir el servicio.
- **Sincronización en Tiempo Real:** Firestore ofrece una integración nativa mediante "*listeners*". Esto es vital para Temporis, ya que permite que si un usuario edita un temporizador en un dispositivo, los cambios se reflejen instantáneamente en cualquier otro dispositivo conectado, algo mucho más costoso de implementar en un entorno SQL tradicional.
- **Gestión de Datos Desconectados (Offline):** Las aplicaciones móviles a menudo enfrentan pérdida de conexión. Firestore gestiona automáticamente la caché local, permitiendo que Temporis siga funcionando sin internet y sincronice los datos de forma transparente al recuperar la conexión.
- **Optimización del Crecimiento:** Al no requerir "*JOINs* (Combinaciones de tablas)" complejos, la velocidad de consulta se mantiene constante independientemente del volumen de datos, garantizando una experiencia de usuario fluida a medida que la base de usuarios de Temporis crezca.

## 3. Seguridad y Control de Acceso: Firebase Security Rules

A diferencia de una base de datos SQL, donde la seguridad suele gestionarse en un servidor intermedio (*o Backend*), Firestore permite definir la lógica de seguridad directamente en la capa de datos mediante *Security Rules* (o Reglas de Seguridad). Esto garantiza que se mantenga la:

- **Privacidad del Usuario:** Se han implementado reglas que validan el “request.auth.uid”. De este modo, un usuario solo puede leer o escribir documentos (temporizadores, registros) cuando el campo `uid` (del temporizador/contador) coincida con su identificador de autenticación.

- **Integridad de los Datos:** Las reglas actúan como un validador de esquemas en tiempo real, impidiendo que se guarden datos malformados o que no cumplan con los tipos definidos en nuestras clases (*User, Timer, etc.*).

- **Seguridad por Defecto:** El sistema sigue el principio de "menor privilegio", denegando cualquier acceso que no esté explícitamente permitido por una regla de seguridad.

# 4. Descripción de Entidades y Atributos

## 4.1. Definición de Entidades y Atributos (*Basado en nuestro código actual*)

### 1. USUARIO (User.java)

- **PK:** uid (String)
- username (String)
- displayName (String)
- email (String)
- password (String)
- profilePhotoUrl (String)
- QrCode (String)
- rol (int)

### 2. TEMPORIZADOR (Timer.java)

- PK: id (String)
- name (String)
- duration (int)
- isActive (boolean)
- createdAt (Timestamp)
- FK: uid (Relación con Usuario)

### 3. REGISTRO_TIEMPO (TimeRegister.java)

- PK: id (String)
- date (long)
- duration (int)
- category (String)
- description (String)
- FK: counterId (Relación opcional con Contador)

### 4. PUBLICACIÓN (Post.kt)

- PK: id (String)
- title (String)
- content (String)
- imageUrl (String)
- webUrl (String)
- createdAt (Timestamp)

### 5. CONTADOR (Counter.java)

(Clase no implementada, aunque mantenida, para futuras implementaciones y para evitar desestabilizar el funcionamiento de la aplicación. Su funcionalidad está siendo implementada mediante TimeRegister.java)

- PK: id (String)
- title (String)
- type (String)
- actualValue (int)
- dailyObjective (int)
- active (boolean)

### 4.2 Relaciones y Lógica de Negocio

A pesar de utilizar un modelo NoSQL, se han mantenido relaciones lógicas para garantizar la integridad:

1. **Usuario - Temporizador (1:N):** Vinculados mediante el `uid` de Firebase Auth.
2. **Temporizador - Registro de Tiempo (1:N):** Permitiendo un historial detallado por actividad.

Por ende, a nivel funcional, se mantienen las siguientes relaciones entre entidades/clases:

- **Usuario - posee - Temporizador (1:N):**
  - Un Usuario puede tener muchos temporizadores.
  - Un Temporizador pertenece obligatoriamente a un Usuario (vía `uid`).
- **Temporizador/Contador - genera - Registro_Tiempo (1:N):**
  - Un Contador puede tener asociados varios registros de tiempo (vía `counterId`).
  - Según tu código, esta relación es opcional (`counterId` puede ser “null (Nulo)”), por lo que la participación del Registro es parcial.

## 5. Notas sobre la Optimización del Modelo

Durante la fase de diseño, se ha procedido a la eliminación de la entidad `Counter.java` (a*unque sigue manteniéndose para futuras implementaciones y para no desestabilizar la integridad y la funcionalidad de nuestro proyecto*), integrando sus funciones en la gestión de Temporizadores. Esta simplificación, sumada a la potencia de Firestore, dota a Temporis de una infraestructura robusta, rápida y preparada para el escalado profesional.